MapReduce и DuckDB поверх RustFS

Вариант 10 «Аналитик»: навыки, пары и тройки навыков, влияние на зарплату, регионы, профилирование

бакет hh-lake • 54 607 вакансий • свой MapReduce = DuckDB (md5 совпадают) • 3 прогона на конфигурацию

1 Краткие итоги

- **Хранилище.** Кластер RustFS из ДЗ №2 (8 папок-дисков, EC 4+4). Создан бакет `hh-lake`, датасет лежит объектом `s3://hh-lake/raw/it_vacancies_2024.csv` (847 619 384 Б, совпадает с исходным файлом).
- **Вариант 10 «Аналитик»** — **54 607** вакансий, из них **33 662** с навыками.
- **Свой MapReduce** читает объект из S3 чанками по 32 МиБ (26 запросов HTTP Range) и считает частоты навыков, пар и троек: **8 369 / 152 112 / 1 098 033** различных сочетаний. Результаты всех трёх размеров лежат в `s3://hh-lake/results/mapreduce/`.
- **DuckDB** считает то же одним SQL по тому же объекту; результаты **полностью совпадают** (все строки в обе стороны и md5 содержимого). Отказ маппера отрабатывается повтором, результат не меняется.
- **Профилирование** (5 конфигураций × 3 прогона, время и пик памяти) и **бонус**: комбайнер и параллельные мапперы ускоряют MapReduce в 1,9 раза; Parquet вместо CSV снижает пик памяти DuckDB в 2,4 раза.
- **Зарплаты и регионы.** Найдены сочетания, повышающие и понижающие медиану; разница объясняется в основном требуемым опытом. Москва платит аналитикам на 36% больше Петербурга и почти вдвое больше Екатеринбурга.

2 Этапы 1–2. Хранилище и загрузка данных

Используется кластер RustFS из ДЗ №2: 8 папок-дисков, схема Erasure Coding 4+4, адрес `127.0.0.1:9000` (подробно о развёртывании и отказоустойчивости — в отчёте ДЗ №2). Для доступа к S3 используется собственный клиент с подписью AWS Signature V4 (`s3client.py`), к которому добавлены чтение диапазона байт (HTTP Range — основа чанкового чтения) и запись байтов.

Создание бакета hh-lake и загрузка датасета (upload_raw.py)

```
$ python upload_raw.py
bucket      : hh-lake
object      : raw/it_vacancies_2024.csv
size bytes  : 847,619,384 (local 847,619,384)
etag        : 45041a115287e29b41fc8f82818ec5a6
sha256 local: 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
OK: object in hh-lake matches the local file
```

2.1 Условие варианта 10 «Аналитик»

В данных у каждой вакансии есть профессиональная роль hh (поле `role`). Вариант задан условием:

> `role` ∈ {«Аналитик», «Системный аналитик», «Бизнес-аналитик», «BI-аналитик, аналитик данных», «Продуктовый аналитик»}.
> «Руководитель отдела аналитики» (1 437 вакансий) **не включён**: это управленческая роль с другим уровнем зарплат.

Роль	Вакансий
Аналитик	30 112
Системный аналитик	10 609
Бизнес-аналитик	8 232
BI-аналитик, аналитик данных	3 969
Продуктовый аналитик	1 685
Итого вариант 10	**54 607**

Нормализация навыков (одинакова в обоих способах): поле `key_skills` делится по «;», у каждого навыка обрезаются пробелы (включая неразрывный) и он приводится к нижнему регистру; пустые и повторы внутри вакансии отбрасываются. В данных каждая вакансия встречается один раз (229 388 уникальных `vacancy_id`), поэтому дедупликация по вакансиям не нужна.

3 Этап 3. Подсчёт навыков, пар и троек

3.1 Способ 1: собственный MapReduce, читающий объект из S3 чанками

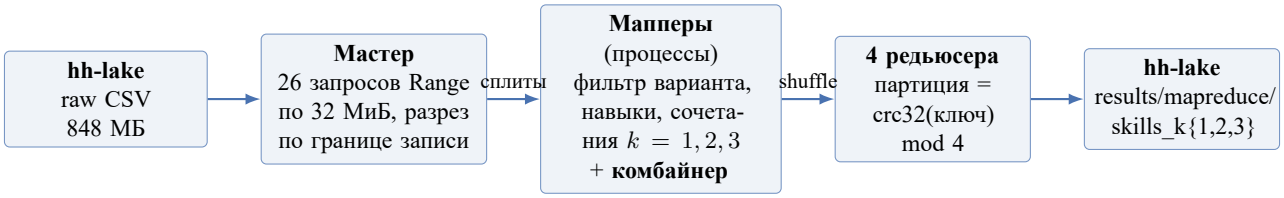


Рис. 1. Схема MapReduce (`mapreduce.py`).

Ключевые решения:

- **Чанки по границе записи.** В CSV есть многострочные поля (описания вакансий в кавычках), поэтому резать по «\n» нельзя. Граница записи — перевод строки, перед которым *чётное* число кавычек; неполная запись переносится в следующий чанк. Функция `last_record_end` проверяется self-check'ом в `__main__`.
- **Партиционирование по crc32**, а не встроенному `hash()`: в Python хеш строк рандомизирован в каждом процессе, и одинаковые ключи из разных мапперов попали бы в разные редьюсеры.
- **Комбайнер** суммирует сочетания внутри сплита до shuffle; флаг `--no-combiner` отключает его для сравнения.
- **Отказ маппера**: флаг `--fail-map-task N` роняет сплит N на первой попытке; мастер перезапускает его (до 3 попыток).

Запуск MapReduce (mapreduce.py)

```
$ python mapreduce.py
splits (range requests): 26, bytes read from S3: 847,619,384
map tasks ok: 26, retries: 0
rows parsed: 229,388, variant rows: 54,607, with skills: 33,662

k=1: distinct=8,369 occurrences=231,216 (sum C(n_i,1) over vacancies = 231,216)
   10653  sql
    8157  аналитическое мышление
    6464  bpmn
    5894  бизнес-анализ
    5226  системный анализ
    5062  анализ данных
    4767  uml
    4452  разработка технических заданий
    4305  работа с большим объемом информации
    3881  аналитика
```

```
k=2: distinct=152,112 occurrences=1,003,894 (sum C(n_i,2) over vacancies = 1,003,894)
    3880  bpmn + uml
    2966  bpmn + sql
    2681  sql + uml
    2671  python + sql
    2507  анализ данных + аналитическое мышление
    2442  sql + системный анализ
    2238  bpmn + системный анализ
    2236  аналитическое мышление + работа с большим объемом информации
    2055  uml + системный анализ
    2050  sql + аналитическое мышление

k=3: distinct=1,098,033 occurrences=3,871,492 (sum C(n_i,3) over vacancies = 3,871,492)
    2201  bpmn + sql + uml
    1699  bpmn + uml + системный анализ
    1387  bpmn + rest + uml
    1266  bpmn + sql + системный анализ
    1236  sql + uml + системный анализ
    1213  rest + sql + uml
    1186  bpmn + rest + sql
    1012  rest + soap + sql
    1002  анализ данных + аналитическое мышление + работа с большим объемом информации
     933  bpmn + uml + бизнес-анализ

timings: map+shuffle 3.69 s, reduce 1.47 s, write results to S3 3.14 s
```

3.2 Способ 2: SQL в DuckDB по тому же объекту

DuckDB читает `s3://hh-lake/raw/it_vacancies_2024.csv` через расширение httpfs. Условие варианта и нормализация навыков написаны на SQL самостоятельно (`duckdb_counts.py`):

Условие варианта и подсчёт троек в DuckDB (фрагмент duckdb_counts.py)

```sql
CREATE TEMP TABLE vacancy_skills AS
WITH variant AS (
    SELECT vacancy_id, key_skills
    FROM read_csv('s3://hh-lake/raw/it_vacancies_2024.csv', header = true, all_varchar = true)
    WHERE role IN ('Аналитик', 'Системный аналитик', 'Бизнес-аналитик',
                   'BI-аналитик, аналитик данных', 'Продуктовый аналитик')
),
exploded AS (
    SELECT vacancy_id,
           lower(trim(unnest(string_split(coalesce(key_skills, ''), ';')),
                      ' ' || chr(9) || chr(10) || chr(13) || chr(160))) AS skill
    FROM variant
)
SELECT DISTINCT vacancy_id, skill FROM exploded WHERE skill <> '';

-- k = 3: тройки навыков внутри одной вакансии, a < b < c
SELECT a.skill AS skill_1, b.skill AS skill_2, c.skill AS skill_3, count(*) AS count
FROM vacancy_skills a
JOIN vacancy_skills b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
JOIN vacancy_skills c ON b.vacancy_id = c.vacancy_id AND b.skill < c.skill
GROUP BY ALL;
```

Запуск DuckDB (duckdb_counts.py)

```
$ python duckdb_counts.py
source: csv; vacancies with skills: 33,662; (vacancy, skill) rows: 231,216

k=1: distinct=8,369 occurrences=231,216
   10653  sql
    8157  аналитическое мышление
    6464  bpmn
    5894  бизнес-анализ
    5226  системный анализ
    5062  анализ данных
    4767  uml
    4452  разработка технических заданий
    4305  работа с большим объемом информации
```

```
    3881  аналитика

k=2: distinct=152,112 occurrences=1,003,894
    3880  bpmn + uml
    2966  bpmn + sql
    2681  sql + uml
    2671  python + sql
    2507  анализ данных + аналитическое мышление
    2442  sql + системный анализ
    2238  bpmn + системный анализ
    2236  аналитическое мышление + работа с большим объемом информации
    2055  uml + системный анализ
    2050  sql + аналитическое мышление

k=3: distinct=1,098,033 occurrences=3,871,492
    2201  bpmn + sql + uml
    1699  bpmn + uml + системный анализ
    1387  bpmn + rest + uml
    1266  bpmn + sql + системный анализ
    1236  sql + uml + системный анализ
    1213  rest + sql + uml
    1186  bpmn + rest + sql
    1012  rest + soap + sql
    1002  анализ данных + аналитическое мышление + работа с большим объемом информации
     933  bpmn + uml + бизнес-анализ

timings: read+filter+explode 0.98 s, k=1 count+write 0.03 s, k=2 count+write 0.36 s, k=3
↪  count+write 1.44 s
```

3.3 Почему число пар и троек растёт так быстро

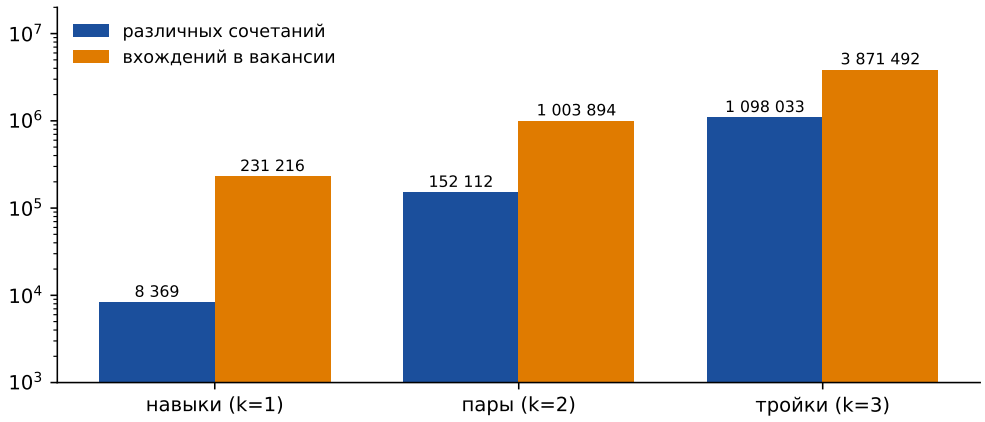


Рис. 2. Различные сочетания и их вхождения в вакансии (логарифмическая шкала).

Вакансия с n навыками даёт $\binom{n}{1} = n$ навыков, $\binom{n}{2}$ пар и $\binom{n}{3}$ троек. Сумма по вакансиям в точности равна числу вхождений, которое получил MapReduce:

$$\sum_i \binom{n_i}{1} = 231\,216, \qquad \sum_i \binom{n_i}{2} = 1\,003\,894, \qquad \sum_i \binom{n_i}{3} = 3\,871\,492.$$

В среднем у вакансии $231\,216/33\,662 \approx 6{,}9$ навыка, но $\binom{n}{3}$ растёт как $n^3/6$: вакансия-«стажёр» с 29 навыками даёт $\binom{29}{3} = 3\,654$ тройки — больше, чем тысяча вакансий по 3 навыка. Поэтому число вхождений растёт в 4,3 раза от навыков к парам и ещё в 3,9 раза к тройкам, а число *различных* сочетаний — в 18 и 7 раз: длинный хвост редких навыков порождает миллион почти уникальных троек (1 098 033 различных на 3,87 млн вхождений).

4 Этап 4. Верификация

`verify.py` читает результаты обоих способов из `hh-lake` и для каждого размера сравнивает *все* строки через `EXCEPT` в обе стороны, плюс контрольные числа: количество ключей, сумма частот и md5 отсортированного содержимого.

Полная сверка MapReduce и DuckDB (verify.py)

```
$ python verify.py
k=1: MapReduce keys=8,369 sum=231,216 md5=9ade43883f080368ed95e9be8c12e0a4
     DuckDB    keys=8,369 sum=231,216 md5=9ade43883f080368ed95e9be8c12e0a4
     rows only in MapReduce: 0, only in DuckDB: 0 -> MATCH
k=2: MapReduce keys=152,112 sum=1,003,894 md5=2a2fd9f522b48fa3b5ddab798a1c6d9d
     DuckDB    keys=152,112 sum=1,003,894 md5=2a2fd9f522b48fa3b5ddab798a1c6d9d
     rows only in MapReduce: 0, only in DuckDB: 0 -> MATCH
k=3: MapReduce keys=1,098,033 sum=3,871,492 md5=076e9954c9f49d38bf94335e7064edcd
     DuckDB    keys=1,098,033 sum=3,871,492 md5=076e9954c9f49d38bf94335e7064edcd
     rows only in MapReduce: 0, only in DuckDB: 0 -> MATCH

RESULT: all three stack sizes are identical
```

Результат: для навыков, пар и троек 0 расхождений в обе стороны и одинаковые md5. Во время сверки нашлась только одна проблема — не в расчёте, а в чтении: автоопределитель формата DuckDB по первым 20 000 строкам решил, что кавычек в файле троек нет, и споткнулся о навык «закупки, продажи, склад». Формат CSV в `verify.py` задан явно.

4.1 Тест отказа маппера

Отказ маппера 5 на первой попытке, повтор и повторная сверка

```
$ python mapreduce.py --fail-map-task 5 --top 3
[master] map task 5 failed (attempt 1): injected failure in map task 5 -> retry
splits (range requests): 26, bytes read from S3: 847,619,384
map tasks ok: 26, retries: 1
rows parsed: 229,388, variant rows: 54,607, with skills: 33,662

k=1: distinct=8,369 occurrences=231,216 (sum C(n_i,1) over vacancies = 231,216)
   10653  sql
    8157  аналитическое мышление
    6464  bpmn

k=2: distinct=152,112 occurrences=1,003,894 (sum C(n_i,2) over vacancies = 1,003,894)
    3880  bpmn + uml
    2966  bpmn + sql
    2681  sql + uml

k=3: distinct=1,098,033 occurrences=3,871,492 (sum C(n_i,3) over vacancies = 3,871,492)
    2201  bpmn + sql + uml
    1699  bpmn + uml + системный анализ
    1387  bpmn + rest + uml

timings: map+shuffle 3.63 s, reduce 1.67 s, write results to S3 3.03 s

$ python verify.py
k=1: MapReduce keys=8,369 sum=231,216 md5=9ade43883f080368ed95e9be8c12e0a4
     DuckDB    keys=8,369 sum=231,216 md5=9ade43883f080368ed95e9be8c12e0a4
     rows only in MapReduce: 0, only in DuckDB: 0 -> MATCH
k=2: MapReduce keys=152,112 sum=1,003,894 md5=2a2fd9f522b48fa3b5ddab798a1c6d9d
     DuckDB    keys=152,112 sum=1,003,894 md5=2a2fd9f522b48fa3b5ddab798a1c6d9d
     rows only in MapReduce: 0, only in DuckDB: 0 -> MATCH
k=3: MapReduce keys=1,098,033 sum=3,871,492 md5=076e9954c9f49d38bf94335e7064edcd
     DuckDB    keys=1,098,033 sum=3,871,492 md5=076e9954c9f49d38bf94335e7064edcd
     rows only in MapReduce: 0, only in DuckDB: 0 -> MATCH

RESULT: all three stack sizes are identical
```

Мастер перехватил исключение сплита №5, запустил его повторно, итог (26 успешных map-задач, 1 повтор) снова совпал с DuckDB до md5. Повтор безопасен, потому что маппер детерминирован и не имеет

побочных эффектов: его результат либо целиком попадает в shuffle, либо отбрасывается.

5 Профилирование и сравнение подходов (+ бонус)

`profile_runs.py` запускает каждую конфигурацию 3 раза отдельным процессом. Время — от старта до завершения процесса; пик памяти — максимум суммарного RSS *всего дерева процессов* (у MapReduce это мастер и воркеры), снимается каждые 50 мс.

Конфигурация	Время, с (3 прогона)	Среднее	Пик, МБ (3 прогона)	Среднее
MapReduce naive (1 worker, no combiner)	20,98 / 20,86 / 20,76	**20,87**	1 240 / 1 242 / 1 222	**1 234**
MapReduce + combiner (1 worker)	16,41 / 16,22 / 16,28	**16,30**	757 / 748 / 751	**752**
MapReduce + combiner (12 workers)	10,90 / 10,89 / 11,10	**10,96**	1 822 / 1 772 / 1 876	**1 823**
DuckDB, CSV from S3	3,21 / 3,69 / 4,05	**3,65**	1 456 / 1 461 / 1 463	**1 460**
DuckDB, Parquet from S3	2,88 / 3,20 / 3,01	**3,03**	599 / 611 / 608	**606**

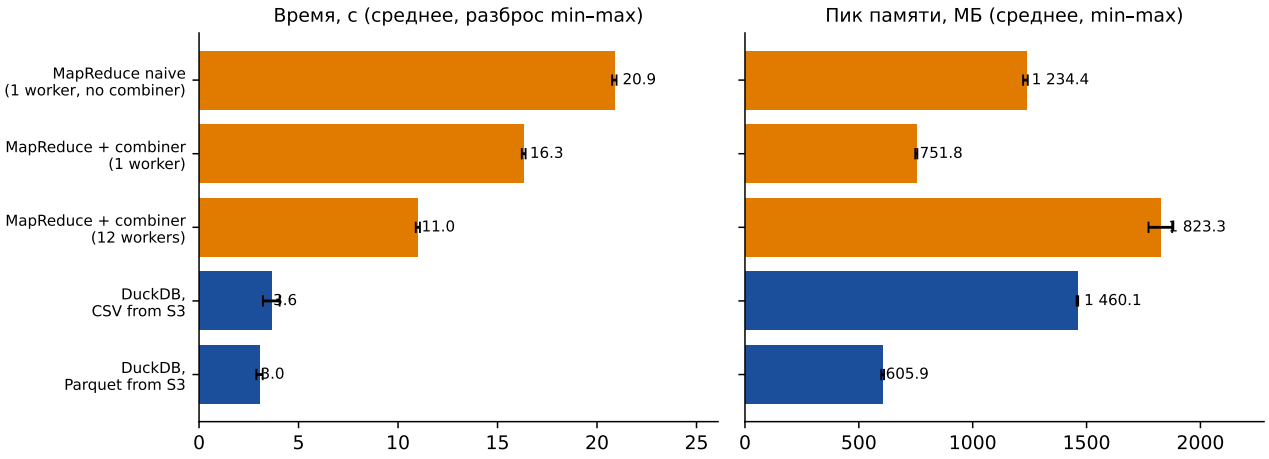


Рис. 3. Время и пик памяти по конфигурациям (оранжевые — MapReduce, синие — DuckDB).

Вывод profile_runs.py

```
$ python profile_runs.py
MapReduce naive (1 worker, no combiner)    run 1:   20.98 s, peak   1239.8 MB, exit 0
MapReduce naive (1 worker, no combiner)    run 2:   20.86 s, peak   1241.8 MB, exit 0
MapReduce naive (1 worker, no combiner)    run 3:   20.76 s, peak   1221.5 MB, exit 0
MapReduce + combiner (1 worker)            run 1:   16.41 s, peak    756.9 MB, exit 0
MapReduce + combiner (1 worker)            run 2:   16.22 s, peak    747.8 MB, exit 0
MapReduce + combiner (1 worker)            run 3:   16.28 s, peak    750.8 MB, exit 0
MapReduce + combiner (12 workers)          run 1:   10.90 s, peak   1821.8 MB, exit 0
MapReduce + combiner (12 workers)          run 2:   10.89 s, peak   1772.0 MB, exit 0
MapReduce + combiner (12 workers)          run 3:   11.10 s, peak   1876.0 MB, exit 0
DuckDB, CSV from S3                        run 1:    3.21 s, peak   1456.5 MB, exit 0
DuckDB, CSV from S3                        run 2:    3.69 s, peak   1460.7 MB, exit 0
DuckDB, CSV from S3                        run 3:    4.05 s, peak   1463.1 MB, exit 0
DuckDB, Parquet from S3                    run 1:    2.88 s, peak    598.6 MB, exit 0
DuckDB, Parquet from S3                    run 2:    3.20 s, peak    610.7 MB, exit 0
DuckDB, Parquet from S3                    run 3:    3.01 s, peak    608.5 MB, exit 0

configuration                          time mean     min     max  peak MB mean      max
MapReduce naive (1 worker, no combiner)        20.87   20.76   20.98        1234.4   1241.8
MapReduce + combiner (1 worker)                16.30   16.22   16.41         751.8    756.9
MapReduce + combiner (12 workers)              10.96   10.89   11.10        1823.3   1876.0
DuckDB, CSV from S3                             3.65    3.21    4.05        1460.1   1463.1
DuckDB, Parquet from S3                         3.03    2.88    3.20         605.9    610.7
```

5.1 Что показывают замеры

- **Комбайнер** (наивный MapReduce → комбайнер, оба в одном воркере): время 20,87 → 16,30 с (−22%), пик памяти 1 234 → 752 МБ (−39%). Без комбайнера маппер отдаёт мастеру каждое вхождение отдельной парой (ключ, 1) — это 231 216 + 1 003 894 + 3 871 492 = 5 106 602 объектов, которые нужно сериализовать, передать между процессами и держать в памяти до свёртки. С комбайнером внутри сплита остаются только различные ключи.
- **12 параллельных мапперов**: время 16,30 → 10,96 с, то есть ускорение всего в 1,5 раза, а пик памяти вырос в 2,4 раза (1 823 МБ): 12 интерпретаторов Python, до 24 сплитов по 32 МиБ в работе и частичные счётчики. Ускорение далеко от 12 раз по закону Амдала: последовательными остаются чтение 26 чанков мастером, свёртка в 4 редьюсерах (≈ 1,5 с), формирование и запись в S3 1,1 млн строк троек (≈ 3 с) и запуск процессов на Windows. Сама map-фаза занимает лишь ≈ 3,6 с из 11.
- **DuckDB против MapReduce**: 3,65 с против 10,96 с у лучшего MapReduce — в 3 раза быстрее. Векторный движок на C++ параллельно разбирает CSV и строит хеш-агрегаты, без сериализации данных между процессами Python. Платит памятью (1 460 МБ): параллельный разбор CSV держит в буферах крупные куски файла, а временные таблицы содержат 1,1 млн троек.
- **Parquet вместо CSV** для DuckDB: время 3,65 → 3,03 с (−17%), пик памяти 1 460 → 606 МБ (−58%). Parquet колоночный и в 5 раз меньше (169 против 848 МБ): DuckDB читает 3 нужные колонки из 23 и не разбирает самое тяжёлое поле — текст описания вакансии. Оставшиеся ≈ 3 с — в основном постоянные затраты (запуск Python, загрузка httpfs, тройное соединение для троек).

Создание Parquet-копии в hh-lake (make_parquet.py)

```
$ python make_parquet.py
CSV     s3://hh-lake/raw/it_vacancies_2024.csv: 847,619,384 bytes
Parquet s3://hh-lake/curated/it_vacancies_2024.parquet: 169,030,666 bytes (5.0x smaller), 229,388
↪  rows, written in 4.3 s
```

После профилирования результаты в `hh-lake` последний раз записали MapReduce на 12 воркерах и DuckDB по *Parquet*; повторная сверка показывает, что и этот путь даёт те же числа:

Сверка после профилирования: MapReduce (12 воркеров) против DuckDB по Parquet

```
$ python verify.py   # after profiling: MapReduce (12 workers) vs DuckDB over Parquet
k=1: MapReduce keys=8,369 sum=231,216 md5=9ade43883f080368ed95e9be8c12e0a4
     DuckDB    keys=8,369 sum=231,216 md5=9ade43883f080368ed95e9be8c12e0a4
     rows only in MapReduce: 0, only in DuckDB: 0 -> MATCH
k=2: MapReduce keys=152,112 sum=1,003,894 md5=2a2fd9f522b48fa3b5ddab798a1c6d9d
     DuckDB    keys=152,112 sum=1,003,894 md5=2a2fd9f522b48fa3b5ddab798a1c6d9d
     rows only in MapReduce: 0, only in DuckDB: 0 -> MATCH
k=3: MapReduce keys=1,098,033 sum=3,871,492 md5=076e9954c9f49d38bf94335e7064edcd
     DuckDB    keys=1,098,033 sum=3,871,492 md5=076e9954c9f49d38bf94335e7064edcd
     rows only in MapReduce: 0, only in DuckDB: 0 -> MATCH

RESULT: all three stack sizes are identical
```

Бонус: «до» и «после». MapReduce: 20,87 с / 1 234 МБ → 10,96 с / 1 823 МБ (в 1,9 раза быстрее, ценой памяти) или 16,30 с / 752 МБ (в 1,3 раза быстрее и на 39% меньше памяти, если ресурс — память). **DuckDB**: 3,65 с / 1 460 МБ → 3,03 с / 606 МБ (на 17% быстрее и в 2,4 раза меньше памяти). Результаты всех вариантов совпадают до md5.

6 Этап 5. Как пары и тройки навыков влияют на зарплату

6.1 Методика

- **Зарплата:** только RUR; середина вилки или единственная граница; «до вычета налогов» приводится к «на руки» (×0,87); выбросы вне [15 тыс.; 1 млн] ₽ отброшены (их 35, минимум был 43,5 ₽). Медиана

варианта — **93 960 ₽** по 18 674 вакансиям.

- **Прибавка пары** = медиана пары − медиана *более частого* навыка пары (база). **Прибавка тройки** = медиана тройки − медиана *самой частой* из трёх её пар (база). Так прибавка показывает, что даёт *дополнительный* навык к распространённому стеку.
- **Надёжность:** сочетание учитывается, если у него ≥ 30 вакансий с зарплатой от ≥ 10 разных работодателей.
- **Повторные публикации** одного объявления (тот же работодатель, название, навыки и вилка) считаются одним предложением — причина ниже. Остаётся 13 811 предложений с зарплатой (9 366 с навыками).

Найденный артефакт данных. В первой версии расчёта таблица «понижающих» пар состояла из десятков сочетаний с `rest` с *одинаковой* медианой 65 250 ₽. Причина: работодатель 6093775 разместил вакансию «Системный аналитик (стажёр)» с вилкой 70–80 тыс. до налогов и списком из 29 навыков в 23 городах; каждая копия давала $\binom{29}{2} = 406$ пар и 3 654 тройки с одной и той же зарплатой. Таких массовых публикаций много:

Работодатель	Вакансия	Копий	Городов	Навыков	На руки, ₽
9498112	Менеджер проектов в сфере медиааналитики	104	62	0	65 000
78638	Консультант Тинькофф Банк	52	52	0	35 670
9498112	Специалист по анализу упоминаний в соцсетях	39	31	0	39 000
6093775	Системный аналитик/System Analysis (стажер)	39	23	29	65 250
9498112	Аналитик в команду WFM	38	38	12	60 000
78638	Консультант Тинькофф Банк	37	37	0	33 060
4132459	Аналитик управления стратегического планирования департамента экономики ЯНАО	36	36	0	102 500
78638	Консультант Тинькофф Банк	35	35	0	30 450

6.2 Пары, которые сильнее всего повышают и понижают медиану

Столбцы: база (более частый навык) и добавленный навык; предложений / работодателей с парой; медиана пары; предложений и медиана базы; прибавка; доля вакансий с опытом 3+ лет у пары / у базы.

База	+ навык	n / раб.	Медиана	n базы	Мед. базы	Прибавка	3+ лет
Повышают							
bpmn	+ **kafka**	42 / 24	297 500	1249	158 340	**+139 160** (+88%)	79% / 45%
sql	+ **kafka**	48 / 28	268 000	1876	135 000	**+133 000** (+98%)	67% / 34%
sql	+ **микросервисная архитектура**	30 / 16	255 500	1876	135 000	**+120 500** (+89%)	97% / 34%
сбор и анализ информации	+ **erp-системы на базе 1с**	90 / 51	200 000	1099	80 000	**+120 000** (+150%)	56% / 21%
сбор и анализ информации	+ **бухгалтерский учет**	30 / 15	200 000	1099	80 000	**+120 000** (+150%)	53% / 21%
uml	+ **kafka**	42 / 24	300 000	830	191 700	**+108 300** (+56%)	81% / 52%
python	+ **a/b тесты**	35 / 25	215 000	546	110 000	**+105 000** (+96%)	43% / 27%
bpmn	+ **микросервисная архитектура**	34 / 20	261 000	1249	158 340	**+102 660** (+65%)	97% / 45%
sql	+ **системная интеграция**	67 / 44	235 000	1876	135 000	**+100 000** (+74%)	76% / 34%
базы данных	+ **xml**	40 / 21	195 000	497	95 000	**+100 000** (+105%)	55% / 25%
Понижают							
rest	+ **работа с базами данных**	31 / 13	100 000	395	220 000	**−120 000** (-54%)	32% / 62%
системный анализ	+ **аналитический склад ума**	30 / 12	65 250	1148	160 000	**−94 750** (-59%)	10% / 45%
uml	+ **оптимизация бизнес-процессов**	48 / 27	102 500	830	191 700	**−89 200** (-46%)	29% / 52%
системный анализ	+ **разработка по**	34 / 14	78 300	1148	160 000	**−81 700** (-51%)	24% / 45%
rest	+ **техническая документация**	42 / 21	140 000	395	220 000	**−80 000** (-36%)	45% / 62%
системный анализ	+ **анализ рынка**	31 / 19	80 000	1148	160 000	**−80 000** (-50%)	19% / 45%
системный анализ	+ **ms excel**	42 / 31	88 500	1148	160 000	**−71 500** (-45%)	10% / 45%
soap	+ **atlassian confluence**	40 / 26	160 000	259	230 000	**−70 000** (-30%)	50% / 60%
sql	+ **аналитический склад ума**	34 / 16	65 250	1876	135 000	**−69 750** (-52%)	3% / 34%
системный анализ	+ **аналитические исследования**	76 / 55	91 350	1148	160 000	**−68 650** (-43%)	30% / 45%

6.3 Тройки, которые сильнее всего повышают и понижают медиану

База (частая пара)	+ навык	n / раб.	Медиана	n базы	Мед. базы	Прибавка	3+ лет
Повышают							
аналитика + сбор и анализ информации	**+ erp-системы на базе 1c**	30 / 19	200 000	452	86 000	**+114 000** (+133%)	60% / 23%
bpmn + uml	**+ kafka**	41 / 23	300 000	673	195 000	**+105 000** (+54%)	80% / 54%
bpmn + sql	**+ kafka**	36 / 20	297 500	521	195 000	**+102 500** (+53%)	75% / 53%
sql + uml	**+ kafka**	36 / 20	300 000	466	200 000	**+100 000** (+50%)	78% / 56%
python + sql	**+ a/b тесты**	33 / 23	220 000	423	120 000	**+100 000** (+83%)	42% / 26%
sql + постановка задач разработчикам	**+ json**	35 / 23	250 000	259	156 600	**+93 400** (+60%)	71% / 42%
sql + базы данных	**+ xml**	37 / 18	195 000	268	104 618	**+90 383** (+86%)	57% / 27%
аналитическое мышление + сбор и анализ информации	**+ erp-системы на базе 1c**	31 / 20	165 300	633	75 000	**+90 300** (+120%)	52% / 16%
bpmn + постановка задач разработчикам	**+ json**	33 / 24	240 000	291	150 075	**+89 925** (+60%)	67% / 47%
аналитическое мышление + моделирование бизнес процессов	**+ erp-системы на базе 1c**	36 / 22	195 700	245	115 000	**+80 700** (+70%)	67% / 31%
Понижают							
sql + системный анализ	**+ информационные технологии**	32 / 13	74 150	463	180 000	**−105 850** (-59%)	28% / 48%
rest + soap	**+ аналитическое мышление**	31 / 15	125 000	208	230 000	**−105 000** (-46%)	29% / 59%
bpmn + sql	**+ python**	45 / 24	100 000	521	195 000	**−95 000** (-49%)	29% / 53%
rest + uml	**+ техническая документация**	37 / 17	135 000	264	230 000	**−95 000** (-41%)	43% / 64%
bpmn + rest	**+ техническая документация**	32 / 18	135 000	258	230 000	**−95 000** (-41%)	41% / 67%
bpmn + rest	**+ базы данных**	31 / 13	135 000	258	230 000	**−95 000** (-41%)	42% / 67%
rest + sql	**+ техническая документация**	30 / 15	135 000	277	225 000	**−90 000** (-40%)	33% / 59%
bpmn + uml	**+ оптимизация бизнес-процессов**	40 / 24	106 875	673	195 000	**−88 125** (-45%)	33% / 54%
rest + soap	**+ бизнес-анализ**	39 / 19	143 550	208	230 000	**−86 450** (-38%)	36% / 59%
rest + sql	**+ базы данных**	32 / 13	140 000	277	225 000	**−85 000** (-38%)	44% / 59%

6.4 Интерпретация

Что повышает зарплату. Самые сильные прибавки дают навыки интеграционной и архитектурной работы, добавленные к базовому стеку системного аналитика:

- **Kafka**: +139 тыс. к BPMN, +133 тыс. к SQL, +108 тыс. к UML; в тройках +100 . . . 105 тыс. к парам BPMN+UML, BPMN+SQL, SQL+UML. Медиана вакансий с Kafka — около 300 тыс. на руки.
- **Микросервисная архитектура** (+120,5 тыс. к SQL) и **системная интеграция** (+100 тыс. к SQL), **XML/JSON** к базам данных и постановке задач (+90 . . . 100 тыс.).
- **A/B-тесты** к Python (+105 тыс.) и к паре Python+SQL (+100 тыс.) — продуктовая аналитика с экспериментами.
- **ERP-системы на базе 1С** и **бухгалтерский учёт** к «сбору и анализу информации» (+120 тыс.): узкая и дорогая ниша внедрения ERP.

Что понижает. Сильнее всего медиану тянут вниз *общие* формулировки, добавленные к технической базе: «работа с базами данных» к REST (−120 тыс.), «аналитический склад ума» к системному анализу (−95 тыс.), «оптимизация бизнес-процессов» к UML (−89 тыс.), MS Excel, анализ рынка, аналитические

исследования (−69 . . . 80 тыс.). В тройках — «информационные технологии», «аналитическое мышление», «техническая документация» к дорогим парам с REST/SOAP (−90 . . . 106 тыс.). Неожиданно понижает и **Python** в тройке с BPMN+SQL (−95 тыс.): такой набор характерен не для системных аналитиков, а для начинающих аналитиков данных и бизнес-аналитиков.

> **Главное объяснение — опыт, а не навык сам по себе.** В последнем столбце таблиц видно: у повышающих пар доля вакансий с опытом от 3 лет 43–97% (у большинства выше 55%) против 21–52% у базы, а у понижающих — 3–50% при 34–62% у базы. Технические интеграционные навыки работодатели перечисляют в вакансиях для опытных системных аналитиков, а общие и «мягкие» формулировки — в вакансиях для начинающих и в массовых объявлениях (например, стажёрские вакансии работодателя 6093775 с медианой 65 250 ₽). Поэтому прибавка — это описание рынка, а не обещание: выучив Kafka без опыта, зарплату в 300 тыс. не получить.

7 Этап 6. Региональный анализ: Москва, Санкт-Петербург, Екатеринбург

Выбраны три крупнейших рынка варианта (26 726, 5 793 и 2 203 вакансии). Регионы сравниваются по всем вакансиям региона (здесь копии вакансии в разных городах законно относятся к разным регионам).

7.1 Ключевые показатели

Регион	Вакансий	С навык.	Навыков	С з/п	P25	Медиана	P75	Удалёнка	Без опыта	3+ лет
Все регионы	54 607	61.6%	6.9	18 674	60 000	**93 960**	150 000	24.2%	10.9%	35.9%
Москва	26 726	60.3%	6.7	6 466	100 000	**150 000**	215 000	24.3%	6.0%	46.2%
Санкт-Петербург	5 793	64.8%	6.8	1 872	75 000	**110 000**	160 000	24.7%	9.0%	38.0%
Екатеринбург	2 203	60.5%	6.7	895	60 000	**80 000**	119 500	25.0%	11.2%	25.6%

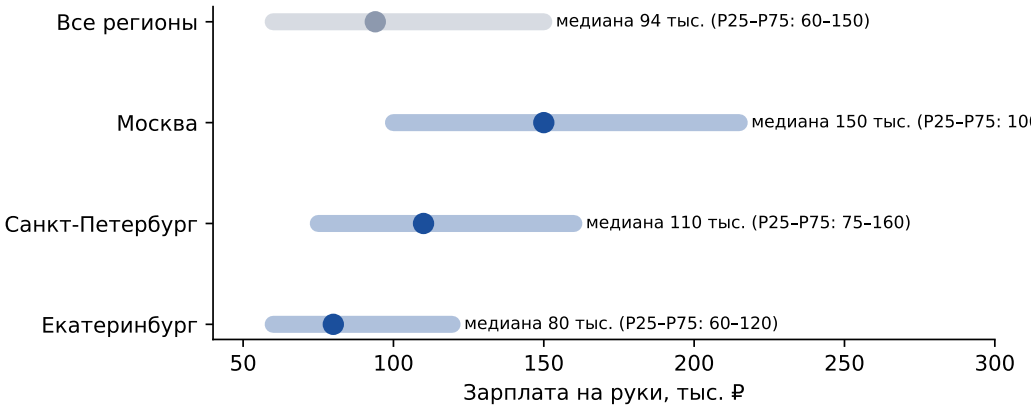


Рис. 4. Медиана и межквартильный размах зарплаты по регионам.

7.2 Самые востребованные навыки, пары и тройки

В ячейке: сочетание; число вакансий, доля от вакансий региона с навыками, медиана зарплаты.

#	Москва	Санкт-Петербург	Екатеринбург
Навыки			
1	sql 6056 вак., 37.6%, медиана 174 000	sql 1144 вак., 30.5%, медиана 125 000	аналитическое мышление 392 вак., 29.4%, медиана 75 000
2	bpmn 3470 вак., 21.5%, медиана 200 000	аналитическое мышление 871 вак., 23.2%, медиана 100 000	sql 367 вак., 27.6%, медиана 104 400
3	аналитическое мышление 2997 вак., 18.6%, медиана 120 000	бизнес-анализ 749 вак., 20.0%, медиана 120 000	бизнес-анализ 264 вак., 19.8%, медиана 90 000
4	бизнес-анализ 2705 вак., 16.8%, медиана 154 425	bpmn 744 вак., 19.8%, медиана 138 730	bpmn 254 вак., 19.1%, медиана 120 000
5	системный анализ 2654 вак., 16.5%, медиана 200 000	системный анализ 615 вак., 16.4%, медиана 150 000	анализ данных 247 вак., 18.5%, медиана 70 000
Пары			
1	bpmn + uml 2166 вак., 13.4%, медиана 225 000	bpmn + uml 454 вак., 12.1%, медиана 150 000	анализ данных + аналитическое мышление 136 вак., 10.2%, медиана 69 000
2	python + sql 1689 вак., 10.5%, медиана 150 000	bpmn + sql 348 вак., 9.3%, медиана 135 000	bpmn + uml 117 вак., 8.8%, медиана 162 500
3	bpmn + sql 1570 вак., 9.7%, медиана 220 000	sql + uml 328 вак., 8.7%, медиана 150 000	sql + аналитическое мышление 110 вак., 8.3%, медиана 76 700
4	sql + uml 1422 вак., 8.8%, медиана 225 000	python + sql 318 вак., 8.5%, медиана 105 000	аналитическое мышление + работа с большим объемом информации 109 вак., 8.2%, медиана 76 700
5	sql + системный анализ 1297 вак., 8.1%, медиана 200 000	sql + системный анализ 300 вак., 8.0%, медиана 145 000	аналитическое мышление + бизнес-анализ 107 вак., 8.0%, медиана 80 000
Тройки			
1	bpmn + sql + uml 1191 вак., 7.4%, медиана 235 000	bpmn + sql + uml 269 вак., 7.2%, медиана 150 000	bpmn + sql + uml 68 вак., 5.1%, медиана 175 000
2	bpmn + uml + системный анализ 899 вак., 5.6%, медиана 239 250	bpmn + uml + системный анализ 214 вак., 5.7%, медиана 162 500	bpmn + uml + системный анализ 64 вак., 4.8%, медиана 82 650
3	bpmn + rest + uml 797 вак., 4.9%, медиана 250 000	bpmn + sql + системный анализ 176 вак., 4.7%, медиана 125 000	sql + анализ данных + аналитическое мышление 60 вак., 4.5%, медиана 74 000
4	bpmn + sql + системный анализ 661 вак., 4.1%, медиана 220 000	sql + uml + системный анализ 176 вак., 4.7%, медиана 145 000	постановка задач разработчикам + разработка технических заданий + системный анализ 55 вак., 4.1%, медиана 160 000
5	bpmn + rest + sql 654 вак., 4.1%, медиана 250 000	bpmn + rest + uml 157 вак., 4.2%, медиана 147 900	анализ данных + аналитическое мышление + работа с большим объемом информации 53 вак., 4.0%, медиана 66 500

7.3 Характерные навыки регионов и цена одинаковых навыков

Регион	Навык	Вакансий	Доля	В двух других	Отношение
Екатеринбург	аналитический склад ума	53	4.0%	1.7%	**2.37**×
Екатеринбург	информационные технологии	38	2.9%	1.3%	**2.15**×
Екатеринбург	работа в команде	66	5.0%	2.4%	**2.06**×
Екатеринбург	1с программирование	34	2.6%	1.4%	**1.77**×
Екатеринбург	анализ бизнес показателей	71	5.3%	3.3%	**1.62**×
Москва	hadoop	225	1.4%	0.4%	**3.28**×
Москва	greenplum	167	1.0%	0.3%	**3.10**×
Москва	dwh	414	2.6%	1.2%	**2.14**×
Москва	vba	184	1.1%	0.6%	**2.00**×
Москва	confluence	178	1.1%	0.6%	**1.76**×
Санкт-Петербург	1с: управление предприятием	70	1.9%	0.7%	**2.54**×
Санкт-Петербург	грамотная речь	106	2.8%	1.2%	**2.44**×
Санкт-Петербург	1с: комплексная автоматизация	80	2.1%	0.9%	**2.38**×
Санкт-Петербург	1с erp	44	1.2%	0.5%	**2.35**×
Санкт-Петербург	1с: управление производственным предприятием	74	2.0%	0.9%	**2.23**×

Навык (8 самых частых)	Москва	Санкт-Петербург	Екатеринбург
bpmn	200 000 (842)	138 730 (218)	120 000 (70)
sql	174 000 (1234)	125 000 (310)	104 400 (95)
uml	217 500 (603)	150 000 (157)	137 500 (36)
анализ данных	120 000 (790)	106 288 (198)	70 000 (119)
аналитическое мышление	120 000 (1130)	100 000 (402)	75 000 (185)
бизнес-анализ	154 425 (840)	120 000 (259)	90 000 (102)
разработка технических заданий	191 400 (643)	150 000 (216)	115 000 (77)
системный анализ	200 000 (691)	150 000 (218)	120 000 (70)

Медиана зарплаты на руки, ₽; в скобках — число вакансий с зарплатой.

7.4 Наблюдения

- **Москва — половина рынка и самые высокие зарплаты.** 26 726 из 54 607 вакансий варианта (49%). Медиана 150 тыс. на руки — на 36% выше Петербурга (110 тыс.) и на 88% выше Екатеринбурга (80 тыс.). Даже нижний квартиль Москвы (100 тыс.) выше медианы Екатеринбурга.
- **Разрыв объясняется и составом вакансий, и ценой навыка.** В Москве больше вакансий для опытных: 46% с опытом от 3 лет против 38% в Петербурге и 26% в Екатеринбурге; без опыта — 6%, 9% и 11%. Но и за *одинаковый* навык Москва платит больше: SQL — 174 / 125 / 104 тыс., системный анализ — 200 / 150 / 120 тыс., UML — 218 / 150 / 138 тыс. Надбавка Москвы за тот же навык — от трети до двух третей.
- **Удалёнка не выравнивает рынок.** Доля удалённых вакансий почти одинакова (24–25% во всех трёх регионах), но зарплаты по-прежнему привязаны к городу публикации.
- **Разный спрос на навыки.** В Москве и Петербурге первый навык — SQL (37,6% и 30,5% вакансий с навыками), в Екатеринбурге — «аналитическое мышление» (29,4%), SQL только второй. Тройка BPMN + SQL + UML — самая частая во всех трёх регионах, то есть ядро системного аналитика общее; различаются окраины.
- **Характерные навыки показывают профиль рынка.** Москва — большие данные и хранилища: Hadoop (в 3,3 раза чаще, чем в двух других регионах), Greenplum (3,1×), DWH (2,1×) — банки и крупные IT-компании. Петербург — внедрение 1С ERP: 1С:УП, 1С:КА, 1С:ERP, 1С:УПП (2,2–2,5×) — производственные и торговые компании. Екатеринбург — общие формулировки («аналитический склад ума», «работа в команде», «информационные технологии») и программирование 1С: вакансии менее специализированы, что согласуется с самой низкой медианой.
- **Чем не различаются.** Доля вакансий с навыками (60–65%) и длина списка навыков (6,7–6,8) почти одинаковы: регионы различаются не количеством требований, а их содержанием и ценой.

8 Ограничения и допущения

- Вариант определён по полю `role`; аналитики с другой ролью (например, «Программист» в названии которого есть «аналитик») не входят.
- Зарплата приводится к «на руки» единым коэффициентом 0,87; прогрессивная шкала НДФЛ не учитывается. Вакансии в других валютах (около 1%) исключены.
- Медианы зарплат — описательная статистика, а не причинный эффект: навык коррелирует с опытом, городом и работодателем (это видно по столбцу «3+ лет»).
- Хранилище — учебный кластер на одном физическом диске (см. ДЗ №2); время чтения из S3 здесь почти равно времени чтения с локального диска.
- Замеры — 3 прогона на одной машине (12 логических ядер, 34 ГБ RAM); пик памяти снимается с шагом 50 мс и может недооценивать кратковременные всплески.

9 Воспроизведение

Код и отчёт: репозиторий GitHub (ссылка в README). Порядок запуска: `upload_raw.py` → `mapreduce.py` → `duckdb_counts.py` → `verify.py` → `mapreduce.py--fail-map-task5` → `analysis.py` → `make_parquet.py` → `profile_runs.py` → `make_figures.py`, `make_tables.py`. Все выводы запусков лежат в `logs/`, таблицы анализа — в `out/` и `s3://hh-lake/results/analysis/`.